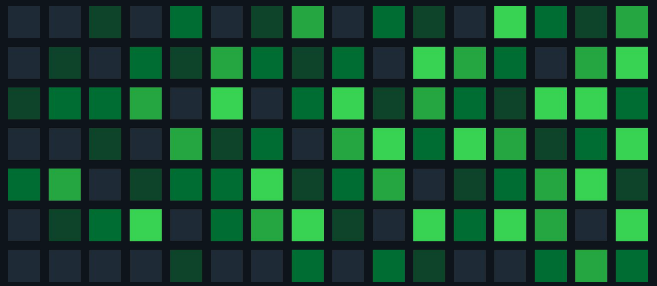
CSSG EDUCATION TEAM · LESSON 4

Public APIs: talking to other programs

Requests, responses, and putting live data on your portfolio



01 · THE IDEA

An API is a contract between two programs

You = the diner

Your app

Wants data it doesn’t have.

→

The menu/waiter

The API

Lists exactly what you can ask for, and how to ask.

→

The kitchen

Their server

Does the work. You never see inside.

You’ve used APIs already: a weather widget, “Sign in with Google,” an embedded map, a Spotify “now playing” bar.

02 · TRY IT NOW

Paste this into your browser’s address bar

```
api.github.com/users/ggreat317
```

```
{
  "login": "ggreat317",
  "avatar_url": "https://avatars.githubusercontent.com/…",
  "html_url": "https://github.com/ggreat317",
  "public_repos": …,
  "followers": …,
  … about 30 more fields
}
```

That’s an API.

No code, no setup. You asked a URL for data and got back JSON: text a program can read.

03 · ANATOMY

Every call is one request and one response

```
GET https://api.github.com/users/ggreat317/repos?sort=updated&per_page=5
```

Method Base URL Endpoint (path) Query parameters

Request: what you send

Method: GET reads data. POST, PUT and DELETE change it.

URL: which resource, plus options.

Headers: extra info, like a key.

Response: what comes back

Status: a number saying how it went.

Headers: info about the reply.

Body: the data, usually JSON.

04 · STATUS CODES

The status code tells you what happened

2xx
It worked

4xx
You asked wrong

5xx
Their server broke

Code	What it means for you
200 OK	Read the body. This is the happy path.
404 Not Found	Typo in the URL, or that user or repo doesn’t exist.
401 / 403	You need a key, or you’ve hit GitHub’s rate limit.
500	Not your fault. Show an error and try again later.

Answer four questions before you write code

Question	GitHub’s answer
1. Which endpoint?	GET /users/{username}/repos
2. What options?	sort, per_page, page
3. Do I need a key?	No, not for public data
4. What’s the limit?	60 requests per hour per IP address

Every API documents these. The layout changes from site to site; the four questions don’t.

06 · IN REACT

Fetch once, store the result in state

```
type Repo = { id: number; name: string; html_url: string };

const [repos, setRepos] = useState<Repo[]>([]);

useEffect(() ⇒ {
  fetch('https://api.github.com/users/ggreat317/repos')
    .then((res) ⇒ res.json())
    .then((data: Repo[]) ⇒ setRepos(data));
}, []);
```

Type what you use

Three fields, not the dozens GitHub sends.

useEffect

Runs after the component renders.

The empty [ ]

Run once, not on every render.

07 · THREE STATES

Every request ends in one of three states

```
useEffect(() => {
  async function load() {
    try {
      const res = await fetch(REPOS_URL);
      if (!res.ok) throw new Error(`GitHub said ${res.status}`);
      setRepos(await res.json());
    } catch (err) {
      setError(String(err));
    } finally {
      setLoading(false);
    }
  }
  load();
}, []);
```

Loading

```
if (loading) return <Skeleton />;
```

Error

```
if (error) return <p>{error}</p>;
```

Data

```
return <RepoList repos={repos} />;
```

The orange line matters: fetch doesn’t throw on a 404.

Higher-level APIs do more of the work for you

Component	`<GitHubCalendar username="ggreat317" />`	One line. Least control.
SDK	`octokit.rest.repos.listForUser({ username })`	Typed functions, no URLs.
fetch	`fetch('https://api.github.com/users/…/repos')`	You build URLs, parse JSON.
HTTP	`GET /users/ggreat317/repos`	What every level becomes.

The trade: going up saves code; going down gives control. Today we used fetch.

09 · ACCESS

Who’s allowed to call it?

Today

Open

Anyone can call it. No sign-up.

GitHub public data, PokéAPI, Open-Meteo

Careful

API key

Sign up, get a key, send it with every request.

OpenWeatherMap, NASA, most paid APIs

Backend lessons

User login

Acts on behalf of a signed-in user (OAuth).

“Sign in with Google,” posting to someone’s GitHub

All three have rate limits. GitHub: 60 requests per hour without a key, 5,000 with one.

10 · HEADS UP

Two walls you’ll hit from the frontend

CORS errors

Some servers don’t let other websites read their responses. Your browser enforces this and blocks the data.

Fix: call it from a server instead.

Keys aren’t secret

All frontend code ships to every visitor, including a key in a VITE_ variable. Anyone can find it in DevTools.

Fix: keep keys on a server.

Both fixes need a backend. That’s where the next lessons go.

11 · LIVE BUILD

Put GitHub on your portfolio, three ways

1 · fetch

Profile card

Avatar, name and repo count from one endpoint.

`/users/ggreat317`

2 · fetch + params

Recent repos

Five most recently updated, with all three states.

`/users/ggreat317/repos`

3 · component

Contribution graph

The green squares, in one line, with a library.

`react-github-calendar`

Same data source, two levels of API. Compare how much code each one took.

HOMEWORK

Add one public API to your portfolio

- ✓ Pick an API that needs no key: GitHub, PokéAPI, Open-Meteo, or your own find.
- ✓ Show data from at least one endpoint, typed with TypeScript.
- ✓ Handle all three states: loading, error and data.
- ☆ **Stretch:** use a higher-level library or SDK for the same API. What did you gain and lose?